

PDblend: Profiling, Online Scheduling, and Adaptive Pool Configuration

PDblend combines mixed execution (M) with prefill/decode disaggregation (P/D) within a GPU cluster. It seeks to reduce serving energy while satisfying time-to-first-token (TTFT) and average time-per-output-token (TPOT) objectives. An *instance* comprises an inference engine and its tensor parallel (TP) GPU group. An M instance executes both stages; a PD request uses a pair of P and D instances. The method has three components: an offline profiler provides cost estimates, an online router assigns requests to execution paths, and an adaptive controller adjusts frequencies, roles, and traffic shares across resident TP pools. The following description reflects the current implementation and identifies boundaries where its execution paths differ.

1 Profiler

The profiler estimates the cost and capacity of an instance under a given model, TP layout, frequency, and request shape. Each topology uses an independent profile because changing TP affects communication, model memory, and KV capacity. Comparisons across frequencies likewise depend on the supported measurement domain. Model, engine, hardware, and measurement identities bind a profile to its intended deployment.

Measurement dimensions. Prefill measurements vary the prompt length L and GPU frequency f . Decode measurements additionally vary batch size B and context length C , recording the shapes actually executed during generation. Mixed measurements observe prefill alongside active decodes, capturing the stalls that long prompts impose on ongoing requests. Collection separates loading, warmup, and clock setup from the measurement windows, and preserves raw observations and independent validation samples. Runtime components also cover idle, parked, and stopped states, wakeup and clock-change latency, KV capacity, and P/D handoff.

Time and power models. Let $T_p(L, f)$ denote prefill time and $T_d(B, C, f)$ denote decode step time. The current native timing model uses a nonnegative fit at each measured frequency:

$$T_p(L, f) = a_f + b_f L + c_f L^2, \quad (1)$$

$$T_d(B, C, f) = u_f + v_f B + w_f BC. \quad (2)$$

The linear terms represent the main workload contribution, while BC captures context-dependent memory access. Normalization used in the implementation is absorbed into the coefficients. Compatibility profiles may also use a B^2 term or piecewise curves; the bound profile version determines the actual query behavior. Power models depend on frequency and batch/context shape.

Where supported by measurements, mixed power comes from a mixed serving window.

Timing and power must refer to consistent observation boundaries. For a serving window of duration H ,

$$E_{\text{window}} = \int_0^H P(t) dt = H\bar{P}, \quad (3)$$

where $\bar{P}$ is its mean power. A native request-cycle mean includes scheduling and intervals between kernels. Multiplying that mean by CUDA-only prefill time would mix incompatible boundaries. Pool energy therefore uses a corresponding cycle or layout-window model. Parked capacity still contributes static power.

Transfer and validated queries. The basic KV cost interface uses a fixed term plus data volume divided by effective bandwidth: $T_x(L) = x_0 + Lk/v$, where k is KV bytes per token and v is effective bandwidth. Other native components interpolate observed values at measured prompt lengths. A difference between M and PD path latency, however, includes protocol and scheduling effects and is not a direct physical-copy measurement. Section 2 instead defines handoff through observable output-token endpoints.

Calibration and tuning data support fitting and selection; independent holdouts check prediction error and selected configurations. Published profiles carry frequency, length, batch/context, and topology domains. Bounded versions reject queries outside their supported coverage. Interpolation does not by itself establish validation for an additional domain. Before planning, the consumer also checks that the required capacity, static-state, transfer, and clock-transition components are available. A profile thus supplies a prediction together with its identity, measurement boundary, and range of applicability. Decisions retain the profile binding for subsequent inspection.

2 Online Algorithm

For each arriving request, the router fixes its prefill and decode owners. Let L be its input length and K its maximum output length. An M route assigns both stages to one instance. A PD route selects an ordered pair (p, d) : p processes the prompt and d continues generation. Both pools may contain multiple instances; the pair is selected for the request, rather than fixed for the entire cluster.

Instance-local token batching remains the engine scheduler's responsibility. The default waiting policy is FCFS with continuous batching and chunked prefill, so admission order does not imply completion order. Within a TP instance, GPUs jointly execute the sched-

uled batch; they do not independently accept requests as individual GPUs become idle.

2.1 Candidate Paths and Instance Pairing

The router reads the published role and admission state of each instance. A PD pair must contain distinct instances with matching model, TP, pipeline parallelism (PP), pool identifier, and generation. The current path supports PP=1 and does not remap KV shapes across different TP layouts. The router therefore enumerates compatible pairs before choosing their members.

When a compatible pair exists, a prompt with $L \geq \tau$ normally prefers PD; other prompts prefer M. If no M instance accepts requests, a short prompt may also use PD. Without a complete PD pair, a long prompt can use M. An enabled pressure policy imposes its own minimum PD input length as well. The threshold τ selects a preferred branch, rather than unconditionally determining the final route.

For instance i , the proxy maintains U_i , the total prompt tokens assigned to requests whose first token has not yet been observed, and N_i , the number of assigned, unfinished decode sequences. Dispatch immediately increases U_p and N_d , allowing later decisions to see future decode load even while prefill is running. These are proxy estimates: U_i is not an exact per-chunk remainder, and N_i is not the engine’s running batch size. The basic load rule compares (N_i, U_i) lexicographically for M and (U_p, N_d, U_d) over compatible PD pairs.

The standard runner enables an additional SLO routing layer. It first checks request length, admission state, and KV capacity for all candidates in the preferred branch. Each endpoint of a PD route is charged the $L + K$ request budget in proxy admission accounting; the engine allocates the actual KV blocks. The router then queries the profile at the current control frequencies. Predicted P queueing sums the prefill costs of waiting requests individually. Decode cost depends on assigned sequences and their context bounds. Summation avoids treating several queued prompts as one fictitious prompt outside the profiling domain.

2.2 Charging the KV Handoff

The proxy first submits the PD request to P. After prefill, P generates a first token, which the proxy forwards to the client. The proxy then submits continuation to D with the carried token, transfer identity, and KV metadata, reducing the remaining output budget to $K - 1$. The connector may submit KV layer by layer during P execution, so physical transfer can start before P emits its token. D continues once the required KV is available. A one-token PD candidate uses only P and does not initiate remote KV transfer.

Let Q_p be P queueing time, $s = T_d(B, C, f_D)$ the D step time, and h the interval from P’s first token to D’s first token. For $K \geq 2$, the ordinary router predicts

$$\text{TTFT}_{PD} = Q_p + T_p(L, f_P), \quad (4)$$

$$\text{TPOT}_{PD} = \frac{h + (K - 2)s}{K - 1}. \quad (5)$$

The interval h includes remaining KV handoff, proxy submission, D queueing and reception, and the first D step. An observed endpoint interval already includes that step; adding s again would double-count it. When $K = 2$, the entire TPOT budget is consumed by this interval. Longer outputs amortize the one-time handoff over subsequent tokens. Prompt length alone is consequently insufficient to judge whether PD is appropriate.

For M, the basic estimate is $Q_m + T_p(L, f_M) + T_d(B, C, f_M)$. The router additionally checks whether an incoming prefill would violate the first-token or remaining output budgets of incumbent requests. This prevents spillover from protecting a new request at the expense of active decodes.

For outputs longer than two tokens, missing endpoint coverage falls back to $h = \max(T_x(L) + s, h_{\min})$. The configured floor $h_{\min}$ is a risk parameter, not measurement evidence. A two-token output requires matching endpoint coverage before PD can be classified as SLO-safe. The current development endpoint component covers batch 1, 16 output tokens, requested P/D clocks of 2520 MHz, and inputs of 512–7168 tokens. It interpolates training maxima; these observations do not validate two-token replies, loaded decode batches, or tail guarantees. Online timestamps record actual hand-off and progress without automatically extending this offline coverage.

2.3 Fallback and Cost-Model Boundaries

Algorithm 1 selects the lowest predicted TTFT among preferred candidates meeting both latency budgets with a safety margin. It spills PD to M only when the preferred PD branch has no safe candidate and a safe M alternative exists. If predictions are unavailable or unsafe, the current implementation may retain a capacity-admitted original path, explicitly marking its SLO status as unproven. Admission is therefore not an unconditional SLO guarantee. Dispatched requests retain their owners. For an abnormal termination after engine submission, uncertain cleanup must be resolved by native acknowledgement before accounting is released.

The endpoint formulation above applies to the ordinary router. Pool-level planning still adds the legacy transfer estimate to PD TTFT. The default ResidentRouter score has no explicit transfer term; its optional SLO/energy context adds transfer to first-step latency and has not adopted Equation (5). With an explicitly

Algorithm 1 Request assignment in the ordinary router

Input: Request (L, K) , roles, profile, and SLO

- 1: Build preferred candidates using τ and compatibility
 - 2: If PD is preferred, also enumerate M alternatives
 - 3: Remove candidates failing length, admission, or KV checks
 - 4: **for** each remaining candidate **do**
 - 5: Predict queueing, compute, handoff, and incumbent risk
 - 6: Mark profile coverage and TTFT/TPOT feasibility
 - 7: **end for**
 - 8: **if** the preferred branch has an SLO-safe candidate **then**
 - 9: Choose the candidate with lowest predicted TTFT
 - 10: **else if** PD is preferred and an SLO-safe M exists **then**
 - 11: Choose the safe M with lowest predicted TTFT
 - 12: **else**
 - 13: Retain an admitted original route; mark SLO unproven
 - 14: Return no route if the original branch has no capacity
 - 15: **end if**
 - 16: Pin (p, d) , update U_p, N_d , and start execution
-

configured, validated incremental-energy estimator, resident routing can compare added joules after coverage, latency, and capacity checks. If any retained alternative lacks energy evidence, the entire set falls back to time scoring. There is no separately validated pure KV-copy energy term; matched serving windows or measured whole-route increments account for its aggregate effect.

3 Adaptive Pool Configuration

While routing handles individual arrivals, adaptation determines capacity over the next planning horizon. The controller uses arrival rates, input/output lengths, and outstanding work to choose P/D/M counts, role frequencies, parked states, and the PD/M threshold. Writing a configuration as c and the current configuration as c_0 , the objective is

$$c^* = \arg \min_{c \text{ feasible}} \{HP(c) + E(c_0, c)\}. \quad (6)$$

Here $P(c)$ includes active and parked power, while $E(c_0, c)$ is measured or estimated transition energy. Feasibility requires latency margins, KV capacity, profile coverage, and preservation of branches owning outstanding requests. Each candidate is charged its own transition cost. If the current configuration remains feasible, a change must exceed a relative saving threshold. The default amortization horizon is 60 seconds; the standard PDblend policy sets the saving threshold to 8%. Runtime configuration may override both.

Transition costs preferably come from paired measurements bound to the source/target configurations and profile. A strict mode excludes uncovered transitions; the ordinary fallback estimates wakeup and clock-change costs without labeling them as measurements. Forecasts retain waiting prefill tokens, remaining de-

code work, and occupied KV, including long requests that have left the recent statistics window. The controller normally observes protection signals every second and replans every ten seconds. Missing coverage leads to retention or conservative recovery rather than qualifying an unsupported energy optimization.

3.1 Frequency Control

Frequency control changes service rate while preserving the model and TP topology. The planner enumerates supported discrete frequencies and selects f_P, f_D, f_M using stage-specific load, latency, and power estimates. P and D may use different clocks, whereas all GPUs within one TP instance receive the same target frequency. The search is discrete enumeration; physical actions only update instances whose state changes.

When the budget-aware Shield is enabled, its fast path identifies the pressured role: PD prefill pressure targets P, decode pressure targets D, and mixed-path pressure targets M. It first raises relevant clocks, then restores capacity under persistent pressure. A brief collective token gap permits a bounded clock probe rather than immediately establishing a capacity shortage. Optional deadline protection searches higher frequencies for the lowest one that removes the predicted risk and maintains an M frequency floor until repeated safe observations permit release. Hold times, cooldowns, and repeated downshift confirmation suppress oscillation.

Hardware actions use physical-GPU locks so concurrent controllers cannot write the same device simultaneously. Blocking operations run in worker threads, allowing urgent clock actions on other instances while one engine loads.

3.2 Role Change

Role control redistributes instances among P, D, and M, subject to the fleet budget and service constraints. Unused instances can remain resident in a low-power state or have their processes stopped. When mapping target counts to concrete instances, the controller first preserves existing roles where possible, then considers residency and load to avoid unnecessary wakeups and reassignment.

An active-to-active role change updates the role and frequency used for subsequent arrivals. Existing requests remain pinned to their original instances, with no migration of live KV. The new role therefore takes effect gradually: an instance can still be finishing work admitted under its previous role. Forecasts retain the original branch ownership of that backlog.

Parking or stopping has a stronger boundary. The controller first disables new admission, waits for proxy-side prefill and decode work to drain, and, when native control is connected, verifies every rank's queues, KV

Algorithm 2 Adaptive pool control

Input: Current configuration, profiles, history, and backlog

- 1: Update forecasts and backlog, preserving owners
 - 2: Observe latency and output progress in the fast loop
 - 3: **if** latency risk is detected **then**
 - 4: Raise clocks; restore capacity for persistent pressure
 - 5: Enforce the protection hold period
 - 6: **end if**
 - 7: **if** replanning is triggered and optimization is allowed **then**
 - 8: Enumerate roles, clocks, parked states, and thresholds
 - 9: Enumerate outer TP shares if joint mode is enabled
 - 10: Filter candidates by coverage, capacity, and SLO
 - 11: Minimize horizon energy plus transition cost
 - 12: Apply saving and stability gates
 - 13: Execute changes while preserving request owners
 - 14: Publish TP shares only after all pools are ready
 - 15: **end if**
 - 16: Record actual state and dispatch counts for feedback
-

blocks, and pending transfers before reducing power or stopping. Wakeup performs startup or resume, readiness checks, clock configuration, and native admission restoration before publishing availability. Actions on different instances can proceed concurrently, but failures prevent affected instances from accepting requests.

The standard policy retains at least $\min(4, N)$ M instances, where N is the pool's instance count. Its default search consequently does not admit arbitrary pure-PD layouts. Lower floors require acceptance evidence matching the model, topology, SLO, and workload domain; specific runners additionally constrain frequencies and reserve recoverable M capacity. This guards against over-consolidation when the queueing approximation cannot fully capture mixed interference.

3.3 TP Weight Change

TP weights denote request shares across two resident pools with different TP sizes. The pools occupy disjoint GPUs and use independent profiles. Online changes

preserve their existing model-shard layouts. An outer search enumerates shares w and $1 - w$; each pool's inner planner chooses roles and frequencies. Their energies are combined using Equation (6). The default share grid has step 0.1 and also includes the current target and the preceding interval's observed allocation.

Future arrival rates are scaled by the candidate shares, but in-flight work remains charged to its original pool. Even a pool assigned zero new traffic retains its full resident-capacity cost, and unused GPUs remain in power accounting. Both inner plans must satisfy coverage, capacity, and SLO constraints. If the current joint configuration is still feasible, the proposal must also meet the saving threshold. New shares are published only after all pools' roles, frequencies, and admission states match the completed plans.

Joint planning requires measured decode and mixed-window power. Shield escalation or a protection hold suspends energy optimization. If any pool fails during execution, the coordinator retains the old target shares and blocks further optimization pending state recovery. Algorithm 2 summarizes the shared control structure; it does not prescribe a fixed sequence in which all three kinds of change must occur.

Dispatch realizes the target through weighted deficits. After A assignments, let pool j have received a_j requests. Among currently feasible pools, it selects

$$j^* = \arg \max_j \{w_j(A + 1) - a_j\}. \quad (7)$$

If the target pool is infeasible, another feasible pool can serve the request. Shares are thus allocation targets, not mandatory rejection rules. Actual dispatch counts feed the next planning interval, exposing deviations caused by capacity constraints. This mechanism does not migrate in-flight requests across TP layouts. Online model-weight resharing remains disabled because a GPU-validated native transaction backend is not yet available.